

EPlanify

RAPPORT DE TESTS

Framework : Jest | Environnement : Node.js

Date : 27 Mars 2026

Total tests	Tests unitaires	Tests d'intégration
118	93	25

1. Résumé Exécutif

Le projet EPlanify dispose d'une suite de tests complète réalisée avec le framework Jest. Au total, 118 cas de test ont été définis répartis en 9 fichiers de tests couvrant l'ensemble des modules critiques de l'application.

Configuration Jest

Environnement : Node.js

Transpilation : Babel (babel-jest)

Couverture : model/**/*.*js, middleware/**/*.*js, routes.js

Pattern de test : **/__tests__/**/*.*.test.js

Version Jest : 30.x

Statistiques globales

Métrique	Valeur
Fichiers de tests	9
Fichiers unitaires	7
Fichiers d'intégration	2
Total cas de test	118
Cas unitaires	93
Cas d'intégration	25
Modules couverts	7 modules

2. Vue d'Ensemble des Suites de Tests

#	Fichier	Type	Module testé	Tests	Grp
1	unit/user.test.js	Unitaire	Modèle User	11	5
2	unit/auth.middleware.test.js	Unitaire	Middleware d'authentification	13	3
3	unit/cours.test.js	Unitaire	Modèle Cours	12	5
4	unit/professeur.test.js	Unitaire	Modèle Professeur	11	6
5	unit/salle.test.js	Unitaire	Modèle Salle	16	7
6	unit/semestre.test.js	Unitaire	Modèles Semestre & JourFerie	15	4
7	unit/affectation.test.js	Unitaire	Modèle Affectation	15	5
8	integration/auth.integration.test.js	Intégration	Authentification	15	5
9	integration/cours.integration.test.js	Intégration	API Cours	10	3
TOTAL				118	43

3.1 User Model

__tests__/_unit/user.test.js | 11 tests

Type : **Unitaire**

Module : Modèle User

% describe('createUser') — 3 test(s)

Description du cas de test

- 1 Devrait créer un nouvel utilisateur avec email et mot de passe hashé
- 2 Devrait créer un utilisateur avec rôle par défaut "user"
- 3 Devrait rejeter les emails en doublon

% describe('getUserByEmail') — 2 test(s)

Description du cas de test

- 1 Devrait retourner un utilisateur par son email
- 2 Devrait retourner null si l'utilisateur n'existe pas

% describe('getUsers') — 2 test(s)

Description du cas de test

- 1 Devrait retourner la liste de tous les utilisateurs sans les mots de passe
- 2 Devrait retourner un tableau vide si aucun utilisateur

% describe('getUserById') — 2 test(s)

Description du cas de test

- 1 Devrait retourner un utilisateur par son ID
- 2 Devrait retourner null si l'utilisateur n'existe pas

% describe('updateUser') — 2 test(s)

Description du cas de test

- 1 Devrait mettre à jour les données d'un utilisateur
- 2 Devrait lancer une erreur si l'utilisateur n'existe pas

3.2 Authentication Middleware

__tests__/_unit/auth.middleware.test.js | 13 tests

Type : Unitaire

Module : Middleware d'authentification

%_ describe('estAuthentifie') — 3 test(s)

#	Description du cas de test
---	----------------------------

- | | |
|---|---|
| 1 | Devrait appeler next() si l'utilisateur est authentifié |
| 2 | Devrait retourner 401 JSON pour les routes API si non authentifié |
| 3 | Devrait rediriger vers /connexion pour les pages si non authentifié |

%_ describe('estAdmin') — 5 test(s)

#	Description du cas de test
---	----------------------------

- | | |
|---|--|
| 1 | Devrait appeler next() si l'utilisateur est authentifié et admin |
| 2 | Devrait retourner 403 JSON pour les routes API si rôle insuffisant |
| 3 | Devrait retourner 401 si non authentifié et c'est une API |
| 4 | Devrait rediriger vers /connexion si non authentifié |
| 5 | Devrait retourner 403 pour les pages si rôle insuffisant |

%_ describe('estResponsableOuAdmin') — 5 test(s)

#	Description du cas de test
---	----------------------------

- | | |
|---|--|
| 1 | Devrait appeler next() si utilisateur est admin |
| 2 | Devrait appeler next() si utilisateur est responsable |
| 3 | Devrait retourner 403 si utilisateur est simple user |
| 4 | Devrait retourner 401 si non authentifié pour API |
| 5 | Devrait rediriger vers /connexion si non authentifié pour page |

3.3 Cours Model

__tests__/_unit/cours.test.js | 12 tests

Type : Unitaire

Module : Modèle Cours

% describe('addCours') — 3 test(s)

Description du cas de test

- 1 Devrait ajouter un nouveau cours avec toutes les données
- 2 Devrait rejeter les codes de cours en doublon
- 3 Devrait créer un cours avec tous les champs requis

% describe('getCours') — 2 test(s)

Description du cas de test

- 1 Devrait retourner la liste de tous les cours
- 2 Devrait retourner un tableau vide s'il n'y a aucun cours

% describe('getCoursById') — 2 test(s)

Description du cas de test

- 1 Devrait retourner un cours par son ID
- 2 Devrait retourner null si le cours n'existe pas

% describe('updateCours') — 2 test(s)

Description du cas de test

- 1 Devrait mettre à jour un cours existant
- 2 Devrait lancer une erreur si le cours n'existe pas

% describe('deleteCours') — 3 test(s)

Description du cas de test

- 1 Devrait supprimer un cours sans affectations
- 2 Devrait rejeter la suppression d'un cours avec affectations
- 3 Devrait lancer une erreur si le cours n'existe pas

3.4 Professeur Model

__tests__/_unit/professeur.test.js | 11 tests

Type : Unitaire

Module : Modèle Professeur

%_ describe('addProfesseur') — 2 test(s)

Description du cas de test

- 1 Devrait ajouter un nouveau professeur avec tous les champs
- 2 Devrait rejeter les matricules en doublon

%_ describe('getProfesseurs') — 2 test(s)

Description du cas de test

- 1 Devrait retourner la liste de tous les professeurs
- 2 Devrait filtrer les professeurs par spécialité

%_ describe('getProfesseurById') — 2 test(s)

Description du cas de test

- 1 Devrait retourner un professeur avec ses relations
- 2 Devrait retourner null si le professeur n'existe pas

%_ describe('updateProfesseur') — 2 test(s)

Description du cas de test

- 1 Devrait mettre à jour un professeur existant
- 2 Devrait lancer une erreur si le professeur n'existe pas

%_ describe('deleteProfesseur') — 2 test(s)

Description du cas de test

- 1 Devrait valider la charge horaire avant suppression
- 2 Devrait retourner erreur si le professeur n'existe pas

%_ describe('Validation des données') — 1 test(s)

Description du cas de test

- 1 Devrait avoir une charge horaire par défaut de 30

3.5 Salle Model

__tests__/_unit/salle.test.js | 16 tests

Type : Unitaire

Module : Modèle Salle

% describe('addSalle') — 3 test(s)

Description du cas de test

- 1 Devrait ajouter une nouvelle salle
- 2 Devrait rejeter les codes en doublon
- 3 Devrait valider la capacité positive

% describe('getSalles') — 2 test(s)

Description du cas de test

- 1 Devrait retourner la liste de toutes les salles triées
- 2 Devrait retourner un tableau vide s'il n'y a pas de salle

% describe('getSalleById') — 2 test(s)

Description du cas de test

- 1 Devrait retourner une salle par son ID
- 2 Devrait retourner null si la salle n'existe pas

% describe('getSallesByType') — 2 test(s)

Description du cas de test

- 1 Devrait filtrer les salles par type
- 2 Devrait retourner un tableau vide si aucune salle du type

% describe('updateSalle') — 2 test(s)

Description du cas de test

- 1 Devrait mettre à jour une salle existante
- 2 Devrait lancer une erreur si la salle n'existe pas

% describe('deleteSalle') — 3 test(s)

Description du cas de test

- 1 Devrait supprimer une salle sans affectations
- 2 Devrait rejeter la suppression d'une salle avec affectations
- 3 Devrait lancer une erreur si la salle n'existe pas

% describe('Validation capacité') — 2 test(s)

Description du cas de test

- 1 Devrait valider les types de salle
- 2 Devrait assurer que capacité est un nombre positif

3.6 Semestre and JourFerie Models

__tests__/_unit/semestre.test.js | 15 tests

Type : Unitaire

Module : Modèles Semestre & JourFerie

%_describe('Semestre Model') — 5 test(s)

Description du cas de test

- 1 Devrait créer un semestre avec dates valides
- 2 Devrait avoir une date de début avant la date de fin
- 3 Devrait pouvoir créer plusieurs semestres différents
- 4 Devrait valider l'ordre chronologique des semestres
- 5 Devrait rejeter les noms en doublon

%_describe('JourFerie Model') — 5 test(s)

Description du cas de test

- 1 Devrait créer un jour férié avec description
- 2 Devrait valider que la date du jour férié est une date valide
- 3 Devrait pouvoir avoir plusieurs jours fériés
- 4 Devrait rejeter les doublons date + semestre
- 5 Devrait calculer correctement les jours fériés d'une année

%_describe('Validations Temporelles') — 3 test(s)

Description du cas de test

- 1 Devrait valider que le semestre couvre une période
- 2 Devrait assurer que les jours fériés sont dans le semestre
- 3 Devrait autoriser les jours fériés en dehors de la période scolaire

%_describe('Relations et Intégrité') — 2 test(s)

Description du cas de test

- 1 Devrait permettre l'association de jours fériés à un semestre
- 2 Devrait permettre la suppression en cascade

3.7 Affectation Model

__tests__/_unit/affectation.test.js | 15 tests

Type : Unitaire

Module : Modèle Affectation

% describe('affecterCoursASalle') — 5 test(s)

Description du cas de test

- 1 Devrait créer une affectation valide (cours + salle + date)
- 2 Devrait rejeter si la salle est déjà occupée à la même plage horaire
- 3 Devrait rejeter si le professeur est déjà assigné à la même plage horaire
- 4 Devrait accepter des plages horaires qui ne se chevauchent pas
- 5 Devrait inclure cours, salle et professeur dans la réponse

% describe('affecterCoursAuSemestre') — 4 test(s)

Description du cas de test

- 1 Devrait créer une affectation récurrente (cours par semaine)
- 2 Devrait rejeter si le semestre n'existe pas
- 3 Devrait valider que jour et semestre sont obligatoires
- 4 Devrait vérifier les conflits sur la première occurrence

% describe('getAffectations') — 1 test(s)

Description du cas de test

- 1 Devrait retourner toutes les affectations

% describe('deleteAffectation') — 2 test(s)

Description du cas de test

- 1 Devrait supprimer une affectation existante
- 2 Devrait retourner erreur si affectation n'existe pas

% describe('Validation des plages horaires') — 3 test(s)

Description du cas de test

- 1 Devrait accepter 08:00-10:00 et 10:00-12:00 (consécutif)
- 2 Devrait rejeter 08:00-10:00 et 09:00-11:00 (chevauchement)
- 3 Devrait gérer les plages sur plusieurs jours différents

3.8 Authentication Integration Tests

__tests__/integration/auth.integration.test.js | 15 tests

Type : Intégration

Module : Authentification

% describe('User Registration Flow') — 5 test(s)

Description du cas de test

- 1 Devrait créer un nouvel utilisateur avec succès
- 2 Devrait valider l'email lors de l'inscription
- 3 Devrait valider la force du mot de passe
- 4 Devrait assigner le rôle par défaut "user" lors de l'inscription
- 5 Devrait rejeter l'inscription avec email en doublon

% describe('User Login Flow') — 3 test(s)

Description du cas de test

- 1 Devrait récupérer un utilisateur par email pour la connexion
- 2 Devrait rejeter la connexion avec email inexistant
- 3 Devrait sécuriser le mot de passe avec bcrypt

% describe('Session Management') — 2 test(s)

Description du cas de test

- 1 Devrait créer une session utilisateur après connexion
- 2 Devrait nettoyer la session à la déconnexion

% describe('Role-based Access Control') — 2 test(s)

Description du cas de test

- 1 Devrait assigner le rôle admin
- 2 Devrait assigner le rôle responsable

% describe('Security Tests') — 3 test(s)

Description du cas de test

- 1 Devrait hasher le mot de passe avant stockage
- 2 Devrait vérifier le mot de passe correctement
- 3 Devrait gérer les injections SQL

3.9 Cours API Integration Tests

__tests__/integration/cours.integration.test.js | 10 tests

Type : Intégration

Module : API Cours

%_ describe('CRUD Operations') — 5 test(s)

#	Description du cas de test
---	----------------------------

- | | |
|---|--|
| 1 | Devrait créer et retourner un nouveau cours |
| 2 | Devrait récupérer tous les cours avec succès |
| 3 | Devrait récupérer un cours par ID |
| 4 | Devrait mettre à jour un cours existant |
| 5 | Devrait supprimer un cours sans affectations |

%_ describe('Validation et Erreurs') — 3 test(s)

#	Description du cas de test
---	----------------------------

- | | |
|---|--|
| 1 | Devrait valider les champs requis lors de la création |
| 2 | Devrait retourner 404 pour un cours inexistant |
| 3 | Devrait empêcher la suppression d'un cours avec affectations |

%_ describe('Contraintes de données') — 2 test(s)

#	Description du cas de test
---	----------------------------

- | | |
|---|---|
| 1 | Devrait rejeter le code en doublon |
| 2 | Devrait assurer la cohérence des données de cours |

4. Analyse par Module

1. User Model (11 tests)

Gestion complète du cycle de vie utilisateur avec validation bcrypt et contraintes uniques sur email.

2. Middleware Auth (13 tests)

RBAC (user/responsable/admin). Distinction routes API (JSON 401/403) vs pages (redirect).

3. Cours Model (12 tests)

CRUD complet avec protection contre suppression de cours ayant des affectations actives.

4. Professeur Model (11 tests)

Filtrage par spécialité, unicité des matricules, chargement des relations (disponibilités, affectations).

5. Salle Model (16 tests)

Module le plus testé. Validation types (Salle/Labo/Classe/Amphithéâtre) et capacités positives.

6. Semestre & JourFerie (15 tests)

Cohérence chronologique, suppression en cascade, unicité des noms de semestres.

7. Affectation Model (15 tests)

Détection de conflits plages horaires (salle et professeur), affectations récurrentes sur semestre.

8. Auth Integration (15 tests)

Flux complet : inscription, connexion, sessions, RBAC, sécurité bcrypt, résistance injection SQL.

9. Cours API Integration (10 tests)

Tests d'intégration API : CRUD, validation champs requis, contraintes données.

5. Conclusion

La suite de tests du projet EPlanify couvre l'ensemble des modules critiques de l'application avec 118 cas de test au total. La stratégie de test combine tests unitaires et d'intégration pour garantir la fiabilité du système.

- **Tests unitaires** : 93 tests — Isolation par mocks Prisma, validation des règles métier.
- **Tests d'intégration** : 25 tests — Flux complets d'authentification, sécurité, sessions.
- **Couverture des erreurs** : Contraintes d'unicité, ressources inexistantes, conflits métier.
- **Sécurité** : Hachage bcrypt, résistance injections SQL, contrôle d'accès RBAC.

Points forts

- Utilisation cohérente des mocks Jest pour isoler la base de données.
- Tests de cas limites (null, tableaux vides, ressources inexistantes).
- Validation métier stricte (plages horaires, contraintes d'affectation).
- Tests de sécurité explicites (bcrypt, injection SQL).

118 tests

93 unitaires + 25 intégration | 9 fichiers | Jest 30.x

